

Introdução



- ◆ Projeto Orientado a Objetos.
- ◆ Unified Modeling Language (UML).

Modelagem do Sistema



- ◆ Necessidade de linguagens para descrever sistemas:
 - * úteis em diversos níveis de abstração;
 - * Compreensíveis dentro e entre organizações.
- ◆ Diagramas de bloco são um começo, mas não cobrem tudo.

Projeto Orientado a Objetos



- ◆ **Projeto Orientado a Objetos (POO)**: uma generalização da programação orientada a objetos.
- ◆ **Objeto** = estado + metodos.
 - * Estado caracteriza e identifica cada objeto.
 - * Métodos oferecem uma **interface abstrata** para os objetos.

Objetos e Classes



- ◆ **Classe**: define o tipo do objeto.
- ◆ Classe define os atributos dos objetos mas o valor dos atributos pode mudar ao longo do tempo.
- ◆ A classe define os métodos utilizados para interagir com todos os objetos daquele tipo.
 - * Cada objeto tem seu próprio estado.
 - * Os métodos definem o comportamento do objeto

Princípios de POO



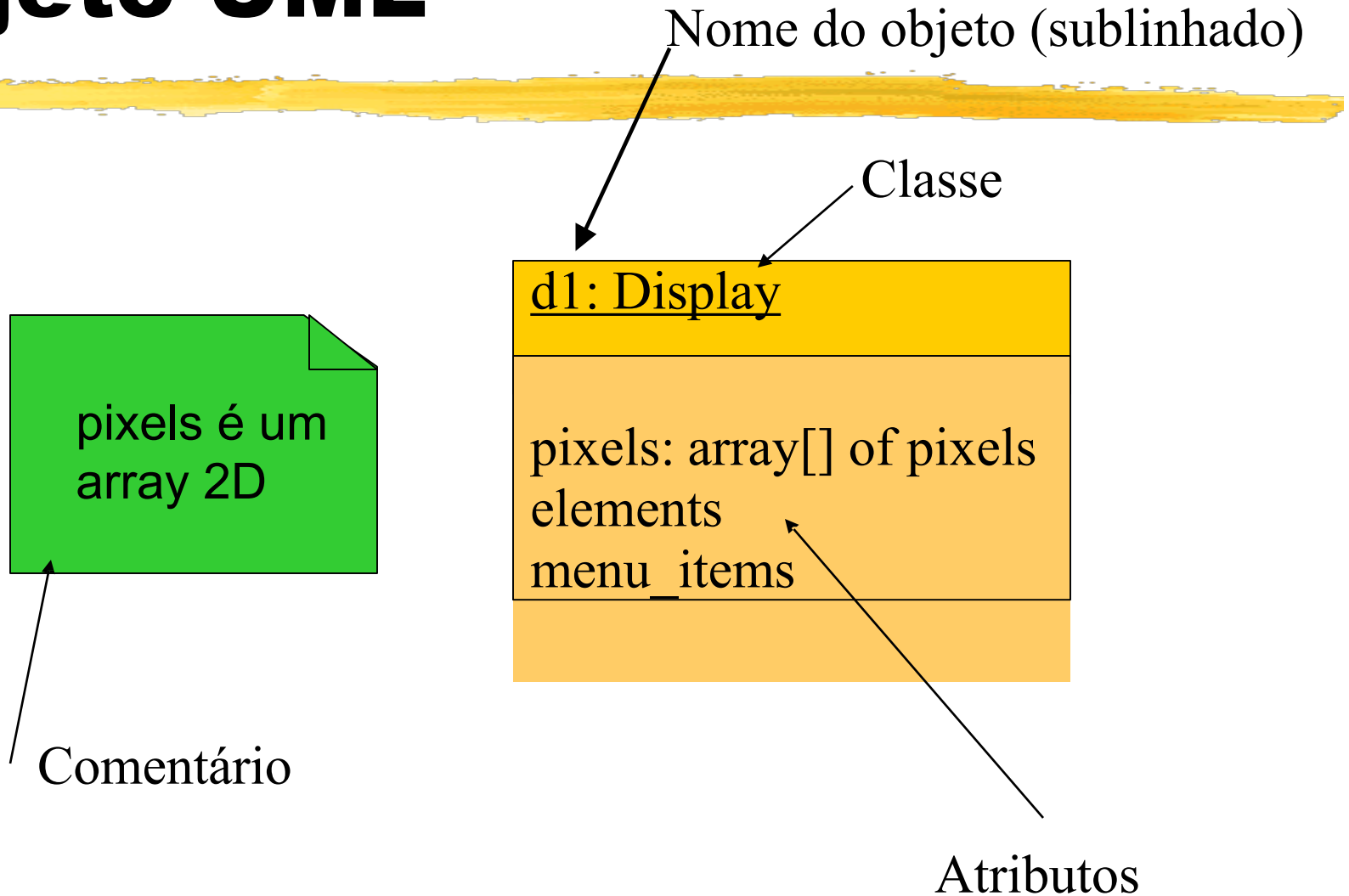
- ◆ Alguns objetos vão corresponder aproximadamente a objetos do mundo real.
 - * Alguns objetos serão úteis apenas para descrição ou implementação.
- ◆ Objetos tem interfaces para ler ou alterar o seu estado, escondendo detalhes de implementação do resto do sistema.

UML: Unified Modeling Language

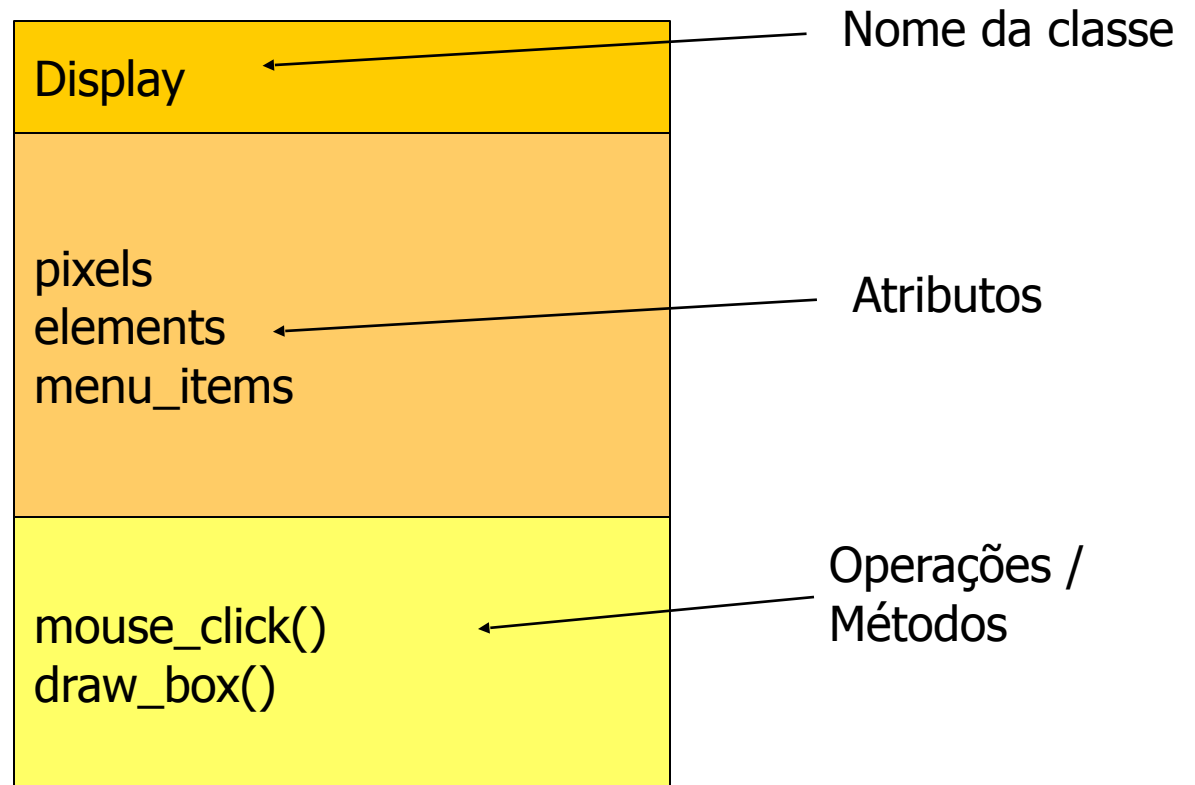


- ◆ Desenvolvida por Booch et al.
- ◆ Características:
 - * Orientada a objetos;
 - * Visual;
 - * Útil para muitos níveis de abstração
 - * Utilizável em todos os aspectos do projeto.

Objeto UML



UML class



A interface da classe

Encapsulamento

- ◆ As operações/métodos fornecem a interface abstrata entre a implementação da classe e outras classes.
- ◆ Operações/métodos podem ter argumentos e retornar valores.
- ◆ Uma operação/método pode examinar e/ou modificar o estado de um objeto.

Escolha apropriadamente a interface



- ◆ Se a interface é **muito pequena/especializada**:
 - * Os objetos podem ser difíceis de utilizar mesmo em uma única aplicação;
 - * Serão mais difíceis ainda de reutilizar.
- ◆ Se a interface é **muito grande**:
 - * A classe se torna muito incômoda para ser compreendida;
 - * A implementação pode ser lenta;
 - * Especificação e implementação tendem a ser mais *bugadas*.

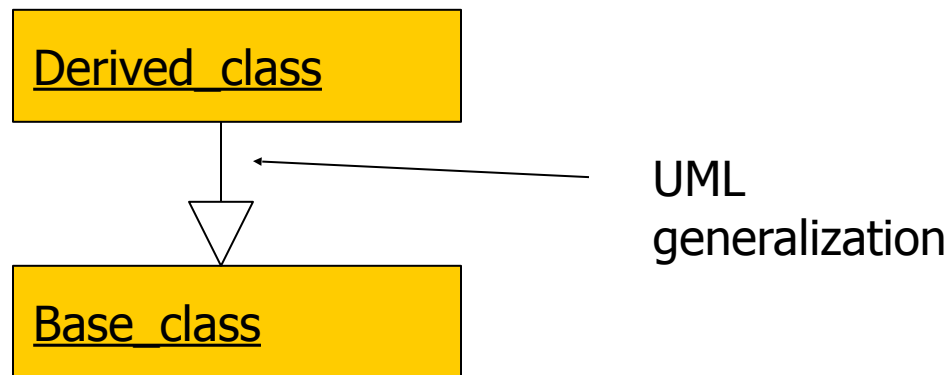
Relacionamentos entre classes e objetos



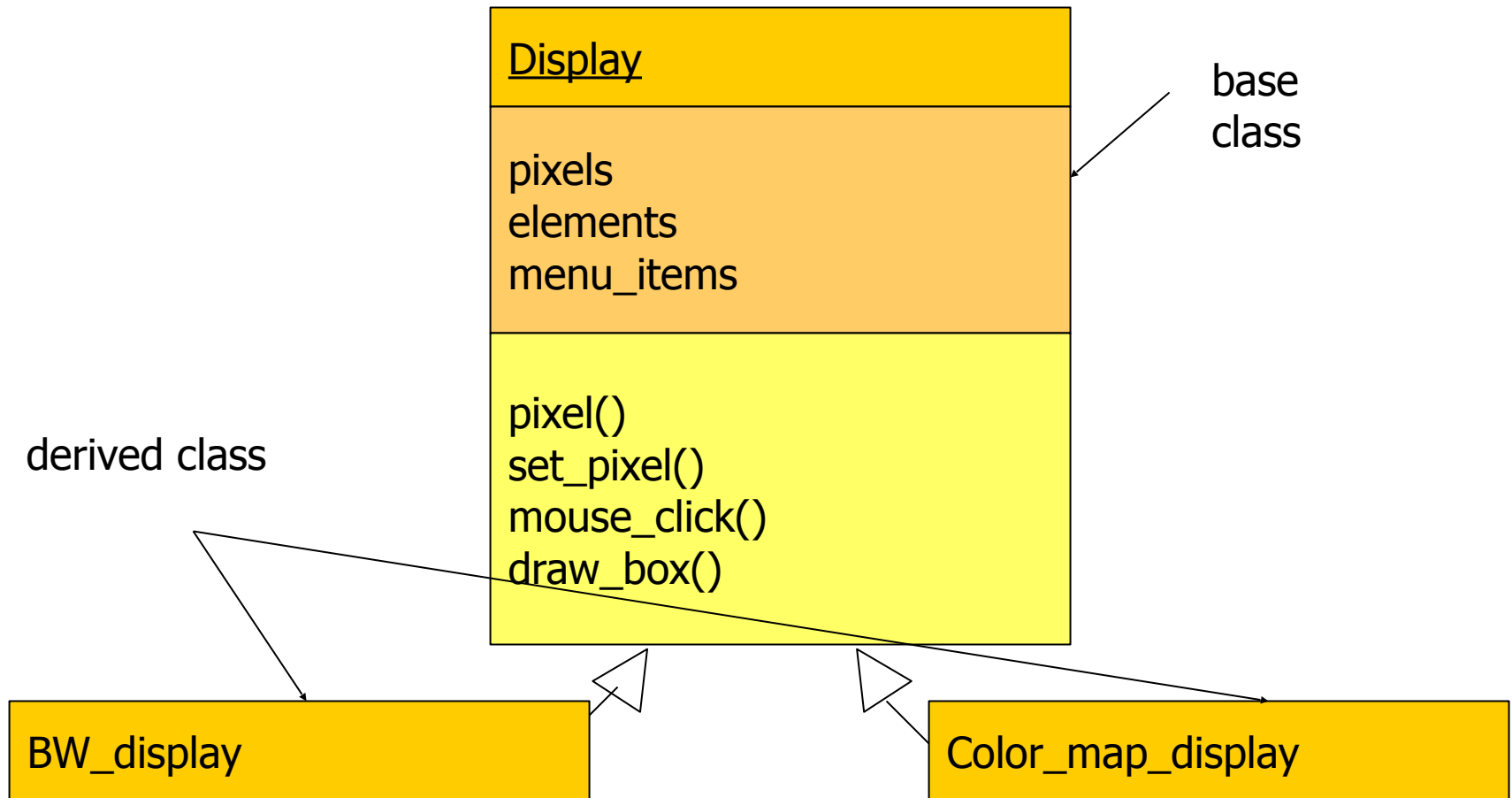
- ◆ **Associação**: objetos comunicam-se mas não um não contém o outro.
- ◆ **Agregação**: um objeto é construído com vários objetos mais simples, que podem existir independente do objeto principal
- ◆ **Composição**: agregação onde os objetos que compõe o todo não existem de forma independente.
- ◆ **Generalização**: define uma classe em termos de outra.

Classes Derivadas

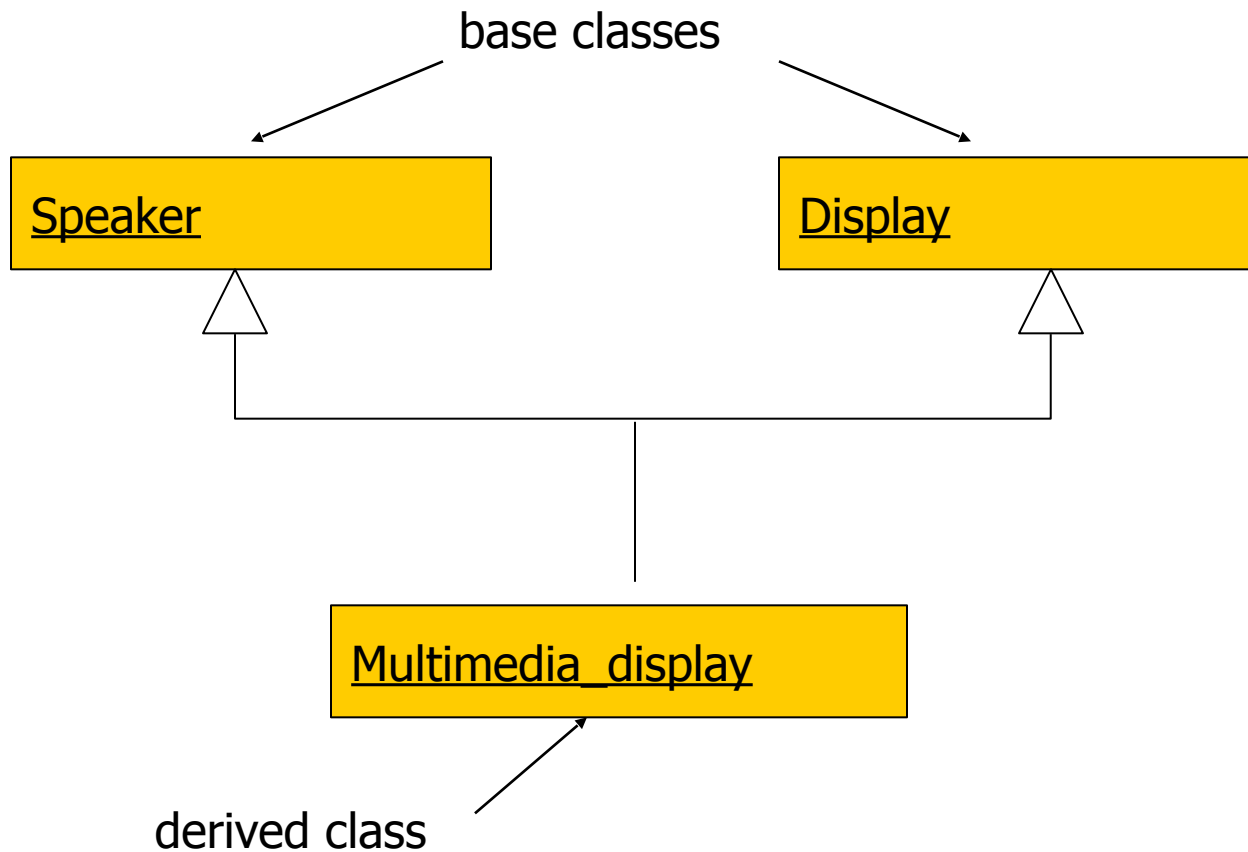
- ◆ Definição de uma classe em função de outra.
 - * A classe derivada **herda** atributos e operações da **classe base**.



Exemplo de Derivação de Classe



Herança Múltipla



Links e associações

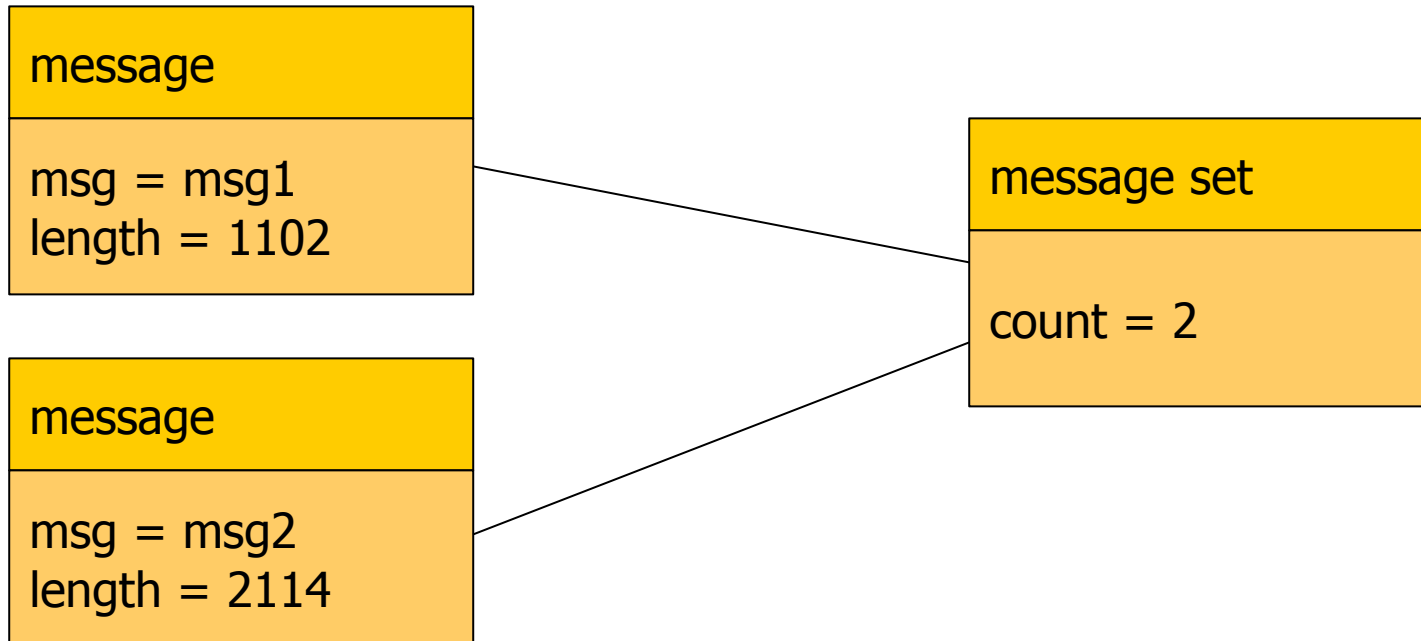


- ◆ **Link**: descreve uma relação entre objetos.
- ◆ **Associação**: descreve relações entre classes.

Link example

Link entre objetos

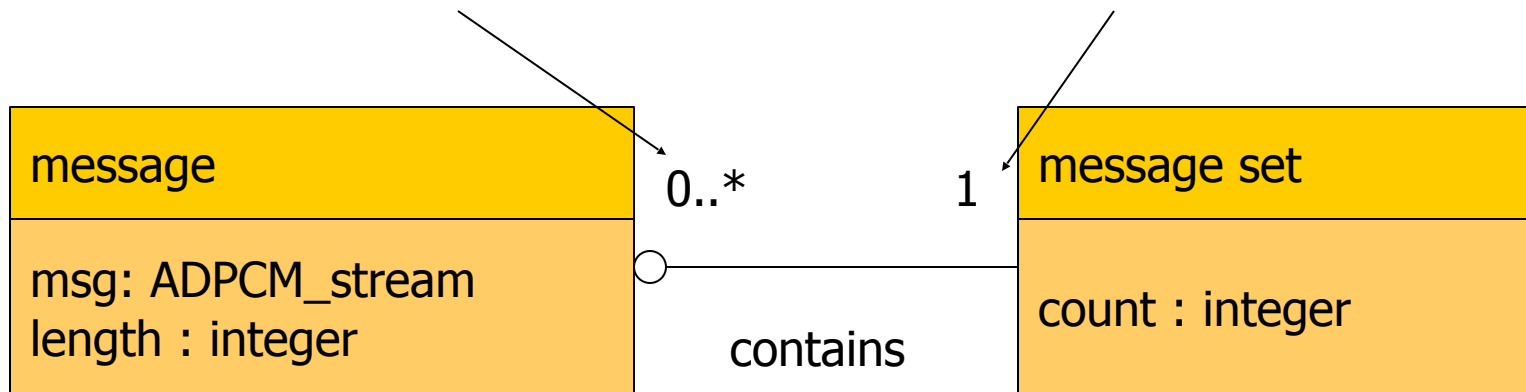
- Link define a relação **contém**:



Exemplo de Associação

contained messages

containing message sets



Associação entre classes

Estereótipos



◆ Estereótipo

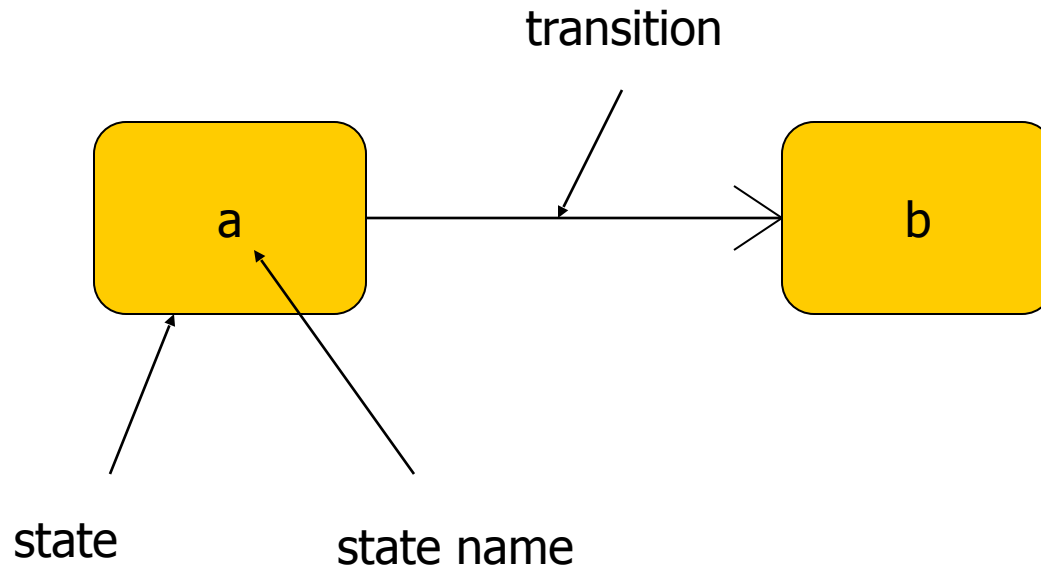
- * É uma notação que permite adicionar semântica extra aos elementos UML, como classes, componentes e associações.
- * Sintaxe: nomes delimitados por “<< >>”
- * Ex: <<signal>>
 - ◆ Indica mecanismo de comunicação assíncrono entre objetos ou sistemas
 - ◆ Sinalizam a ocorrência de eventos

Descrição Comportamental



- ◆ Modela o comportamento dinâmico de um sistema, i.e., como seus elementos interagem e evoluem ao longo do tempo
- ◆ Existem diversas maneiras de se descrever o comportamento:
 - * Visão interna;
 - * Visão externa.

Máquinas de Estado - FSM



FSM Baseadas em Eventos



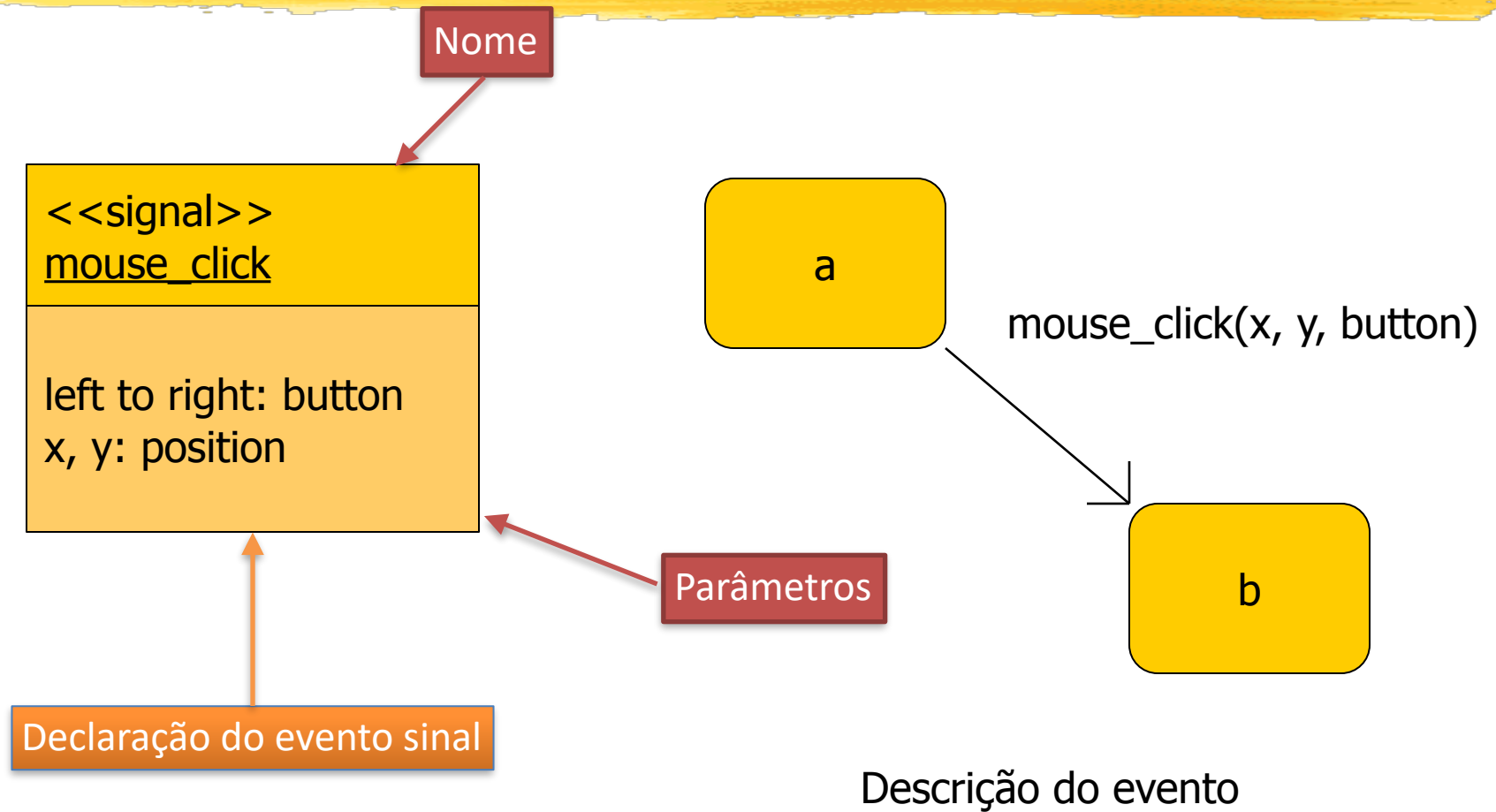
- ◆ Descrições comportamentais podem ser descritas como máquinas de estado orientadas a eventos.
 - * A máquina muda de estado quando recebe uma entrada (sinalizando evento).
- ◆ Um evento pode vir de dentro ou de fora do sistema.

Tipos de Eventos



- ◆ **Sinal**: evento assíncrono.
- ◆ **Call**: comunicação sincronizada.
- ◆ **Timer**: ativado pelo tempo.

Evento Sinal

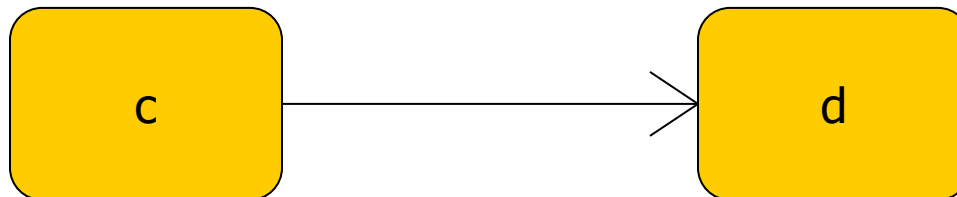


Evento Chamada

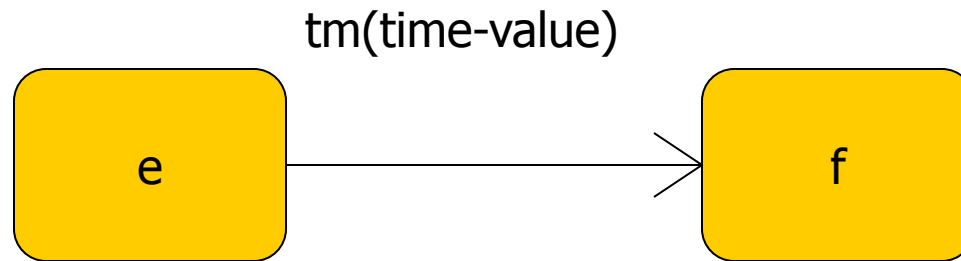
<<call>>
draw_box

x, y: position
dx, dy: dimensions
color: rgb

draw_box(10, 5, 3, 2, blue)



Evento Timer (time-out)



Exemplo Máquina de Estados

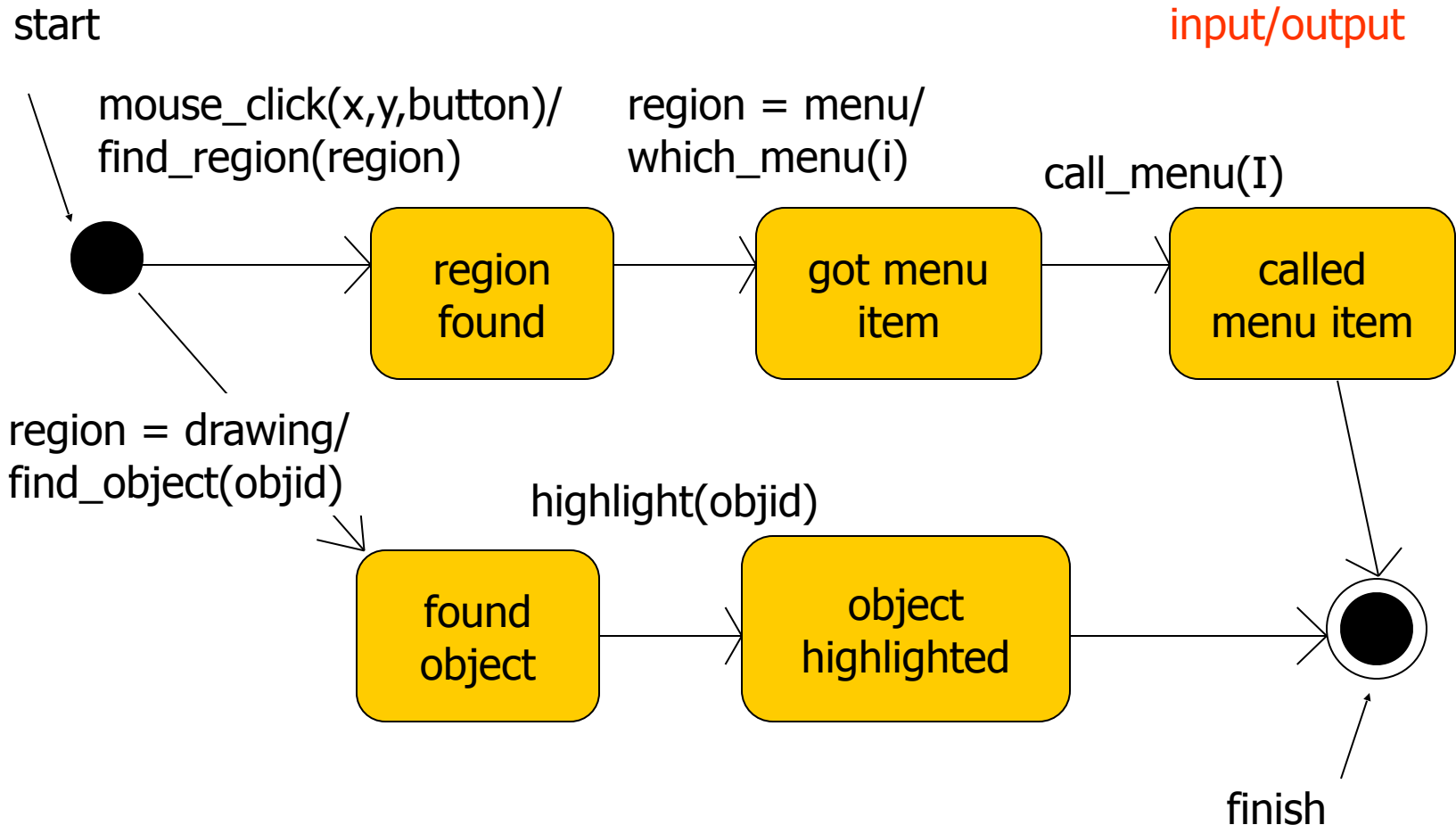


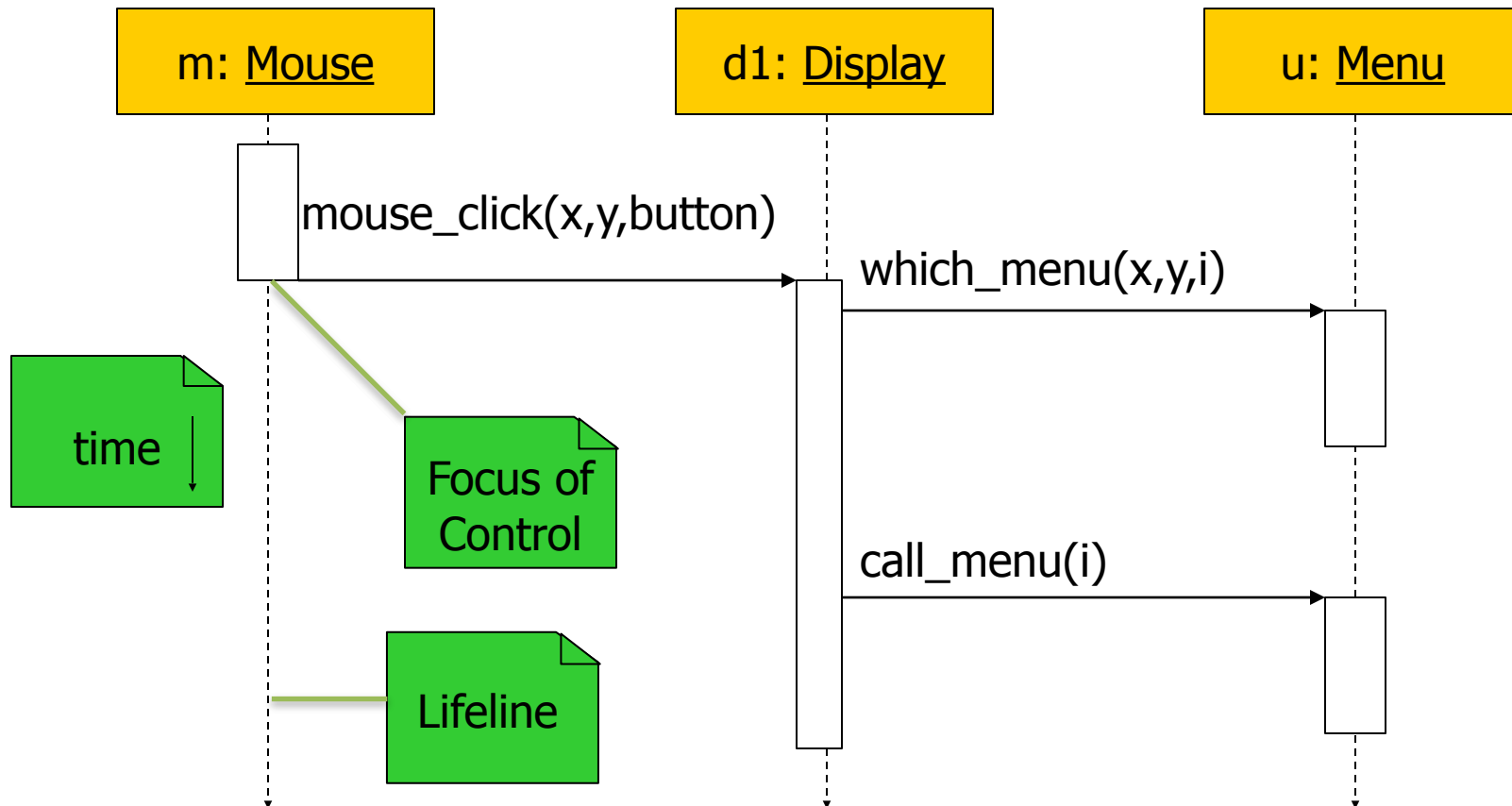
Diagrama de Sequência



- ◆ Mostra uma sequencia de operações ao longo do tempo.
- ◆ Relaciona comportamentos de múltiplos objetos.

Exemplo Diagrama Sequência

Clica no menu



Em suma...



- ◆ A orientação a objetos ajuda a organizar o projeto
- ◆ UML é uma linguagem de descrição de sistemas flexível.
 - * Oferece descrições estruturais e comportamentais.
 - * Pode ser utilizada em diferentes ambientes, linguagens e plataformas